

Course Number	ELE 888
Course Title	Intelligent Systems
Semester/Year	Winter 2026
Instructor	Dr. Mohammed Saif
Section Number	12

Lab 1

Report Title	Lab 1
Submission Date	
Due Date	02/02/2026

Student Name	Student ID	Signature*
Habiba Nagy	501133261	H.N

(Note: Remove the first 4 digits from your student ID)

**By signing above, you attest that you have contributed to this submission and confirm that all work you have contributed to this submission is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a “0” on the work, an “F” in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at: <http://www.ryerson.ca/senate/policies/pol60.pdf>.*

Introduction

In this laboratory experiment, Bayesian decision theory was used to design a probabilistic classifier for a binary classification problem. The objective was to distinguish between two flower species, Setosa and Versicolour, using features from the Iris dataset. By modeling each class with Gaussian probability density functions, a minimum-error-rate classifier was implemented and evaluated.

Beyond basic classification, the lab also explored how decision outcomes change when misclassification costs are not equal. This allowed an examination of risk-based decision making and demonstrated how Bayesian classifiers can be adapted to prioritize certain types of errors over others. Through this process, the lab provided practical experience with probabilistic modeling, decision boundaries, posterior probabilities, and cost-sensitive classification.

Laboratory Exercise

Discriminant Function

The discriminant function compares the weighted likelihoods of the two classes using Bayes' rule. Taking the logarithm simplifies computation and avoids numerical underflow. The sign of the discriminant function determines the class decision.

$$g(x) = p(x|w_1)P(w_1) - p(x|w_0)P(w_0)$$

The decision rule is simple:

- If the discriminant value is greater than zero, the sample is classified as Versicolour.
- Otherwise, the sample is classified as Setosa.

This discriminant function forms the basis of the minimum-error-rate classifier when misclassification costs are assumed to be equal.

Minimum-Error-Rate Classifier using Python

Under the assumption of equal misclassification costs, the Bayes decision rule minimizes the overall probability of classification error. Using the discriminant function, the classifier evaluates each feature value and assigns it to the class with the higher posterior likelihood.

This equal-cost classifier serves as the baseline model for the experiment. Its decision boundary represents the optimal threshold that balances errors between the two classes without favoring one type of mistake over another.

Code:

```
def classify_min_error(x):  
    return 1 if g(x) > 0 else 0
```

Posterior Probabilities + $g(x)$ using Python

In addition to producing class labels, posterior probabilities were computed for both classes given a feature value. These probabilities represent the likelihood that a sample belongs to each class after observing the feature.

Reporting posterior probabilities provides insight into the confidence of the classifier. Feature values near the decision boundary typically produce posterior probabilities closer to 0.5, indicating uncertainty, while values far from the boundary result in more confident predictions.

Code:

```
def gaussian_pdf(x, mu, sigma):  
    return (1 / np.sqrt(2 * np.pi * sigma**2)) * \  
        np.exp(-0.5 * ((x - mu) / sigma)**2)  
  
mu0 = 5.01  
sigma0 = 0.35  
mu1 = 5.94  
sigma1 = 0.52  
P0 = 0.5 # P(w0)  
P1 = 0.5 # P(w1)  
  
def bayesian_classifier(x):  
    """
```

Input:

x : feature value

Output:

P_{w0_x} : posterior probability $P(w_0 | x)$

P_{w1_x} : posterior probability $P(w_1 | x)$

g_x : discriminant function $g(x)$

"""

```
px_w0 = gaussian_pdf(x, mu0, sigma0)
```

```
px_w1 = gaussian_pdf(x, mu1, sigma1)
```

```
px = px_w0 * P0 + px_w1 * P1
```

```
Pw0_x = (px_w0 * P0) / px
```

```
Pw1_x = (px_w1 * P1) / px
```

```
gx = (px_w1 * P1) - (px_w0 * P0)
```

```
return Pw0_x, Pw1_x, gx
```

```
test_values = [3.3, 4.4, 1.2, 5.0, 5.7, 6.3, 1.5]
```

```
print("x\tP(w0|x)\tP(w1|x)\tg(x)")
```

```
for x in test_values:
```

```
    Pw0, Pw1, gx = bayesian_classifier(x)
```

```
    print(f'{x:.1f}\t{Pw0:.3f}\t{Pw1:.3f}\t{gx:.4f}')
```

Optimal Threshold μ_1 (Equal Cost)

The optimal decision threshold for the equal-cost case was determined by identifying the feature value at which the discriminant function crosses zero. This point represents the boundary where both classes are equally likely under the Bayesian model.

The threshold was found numerically by evaluating the discriminant function across a range of feature values. A plot of the discriminant function was generated, and the threshold was marked as a vertical line to clearly visualize the decision boundary.

Code:

```
x_vals = np.linspace(min(class0.min(), class1.min()),
                      max(class0.max(), class1.max()), 2000)

g_vals = np.array([g(x) for x in x_vals])

idx = np.argmin(np.abs(g_vals))

mu1 = x_vals[idx]

plt.figure()

plt.plot(x_vals, gaussian_pdf(x_vals, mu0, sigma0), label="Setosa")
plt.plot(x_vals, gaussian_pdf(x_vals, mu1, sigma1), label="Versicolour")
plt.axvline(mu1, linestyle='--', label=f" $\mu_1 = \{\text{mu1:.2f}\}$ ")

plt.xlabel("Feature value")
plt.ylabel("Probability density")

plt.legend()

plt.show()
```

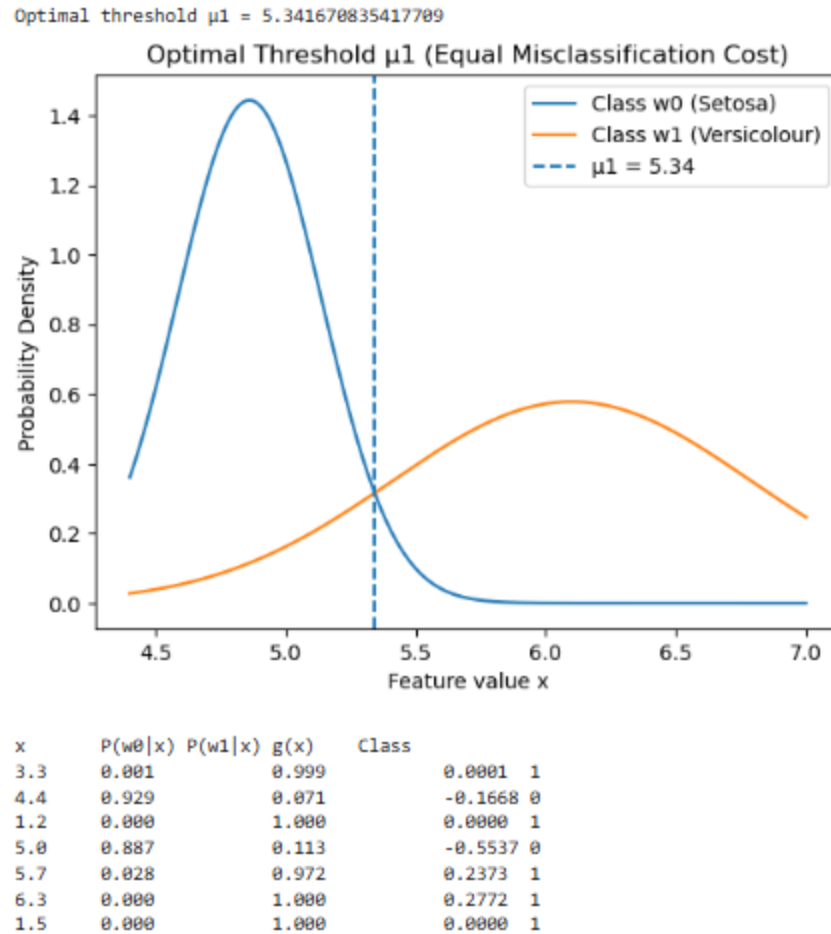


Figure 1.0

Figure 1.0 shows us the results of the previous code, which shows us that the optimal threshold value μ_1 is approximately 5.34.

Second Threshold μ_2 (High Penalty)

The second threshold μ_2 , when a high penalty is assigned for misclassifying w_1 as w_0 . Plot, and report the threshold μ_2 with different penalties.

Code:

```
L_high = np.array([[0, 10],
                  [1, 0]])
```

```
R0_list, R1_list = [], []
```

```

for x in x_vals:
    R0, R1, _ = risk_based_decision(x, L_high)
    R0_list.append(R0)
    R1_list.append(R1)

R0_list = np.array(R0_list)
R1_list = np.array(R1_list)

idx2 = np.argmin(np.abs(R0_list - R1_list))
mu2 = x_vals[idx2]

plt.figure()
plt.plot(x_vals, gaussian_pdf(x_vals, mu0, sigma0), label="Setosa")
plt.plot(x_vals, gaussian_pdf(x_vals, mu1, sigma1), label="Versicolour")
plt.axvline(mu2, linestyle='--', label=f" $\mu_2 = \{\mu_2:.2f\}$ ")
plt.legend()
plt.show()

```

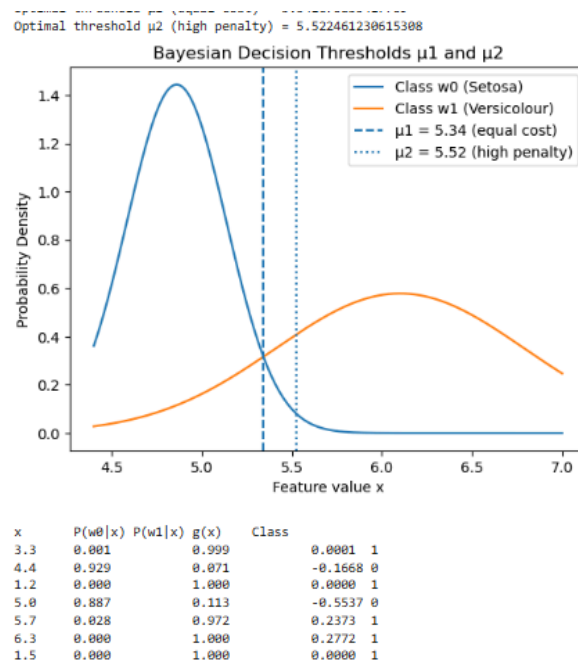


Figure 2.0

Figure 2.0 shows that the optimal threshold μ_2 is 5.22

Table for Test Values (Sepal Length)

Using the Bayesian classifier developed in the previous steps, posterior probabilities and discriminant function values were computed for a predefined set of test feature values using sepal length as the input feature. For each test value x_1 , the posterior probabilities $P(w_0|x_1)$, and $P(w_1|x_1)$, along with the discriminant function value $g(x_1)$, were calculated. The final class label was determined based on the sign of the discriminant function, where a positive value indicates Class w_1 and a negative value indicates Class w_0 .

Code:

```
def gaussian_pdf(x, mu, sigma):  
    return (1 / np.sqrt(2 * np.pi * sigma**2)) * \  
        np.exp(-0.5 * ((x - mu) / sigma)**2)  
  
mu0 = 5.01  
sigma0 = 0.35  
mu1 = 5.94  
sigma1 = 0.52  
P0 = 0.5  
P1 = 0.5  
  
def g(x):  
    return gaussian_pdf(x, mu1, sigma1) * P1 - \  
        gaussian_pdf(x, mu0, sigma0) * P0  
  
def posteriors(x):  
    p0 = gaussian_pdf(x, mu0, sigma0) * P0  
    p1 = gaussian_pdf(x, mu1, sigma1) * P1  
    total = p0 + p1  
    return p0 / total, p1 / total  
  
x_test = [3.3, 4.4, 1.2, 5.0, 5.7, 6.3, 1.5]
```

```

print("x\tP(w0|x)\tP(w1|x)\tg(x)\tClass")

for x in x_test:

    Pw0, Pw1 = posteriors(x)

    gx = g(x)

    label = "w1" if gx > 0 else "w0"

    print(f"{x:.1f}\t{Pw0:.3f}\t{Pw1:.3f}\t{gx:.4f}\t{label}")

```

x	$P(w_0 x)$	$P(w_1 x)$	$g(x)$	Class
3.3	0.794	0.206	-0.0000	w0
4.4	0.963	0.037	-0.1200	w0
1.2	0.000	1.000	0.0000	w1
5.0	0.884	0.116	-0.4948	w0
5.7	0.191	0.809	0.2632	w1
6.3	0.002	0.998	0.3012	w1
1.5	0.000	1.000	0.0000	w1

Figure 3.0

Table for Test Values (Sepal Width) + Feature Comparison

The Bayesian classifier was next evaluated using sepal width as the input feature. Posterior probabilities, discriminant function values, and class labels were computed for the same set of test feature values used previously. The results were then compared with those obtained using sepal length to assess which feature provides better separation between classes w_0 (Setosa) and w_1 (Versicolour).

x	$P(w_0 x)$	$P(w_1 x)$	$g(x)$	Class
3.3	0.768	0.232	-0.3459	w0
4.4	1.000	0.000	-0.0202	w0
1.2	0.010	0.990	0.0000	w1
5.0	1.000	0.000	-0.0001	w0
5.7	1.000	0.000	-0.0000	w0
6.3	1.000	0.000	-0.0000	w0
1.5	0.009	0.991	0.0001	w1

Figure 4.0

Observations

When sepal length was used as the feature, the classifier generally made confident decisions. For smaller values of the feature, the posterior probability strongly favored Class w_0 , while larger values clearly favored Class w_1 . Around the decision boundary, the probabilities were more balanced, which is expected since the two classes overlap in that region. The optimal threshold for the equal-cost case was found to be around $\mu_1 = 5.34$.

After introducing a high penalty for misclassifying Class 0 as Class 1, the decision boundary shifted slightly to the left, to approximately $\mu_1 = 5.22$. This change shows that the classifier became more cautious when assigning Class 1, preferring to reduce costly mistakes even if it meant increasing less severe errors. As a result, some feature values were assigned different class labels compared to the no-penalty case.

When the same process was repeated using sepal width, the results were less decisive. The posterior probabilities were often closer to each other, and the discriminant values were smaller, indicating more overlap between the two classes.

Conclusion

This lab showed how Bayesian decision theory can be used to classify data based on probability rather than fixed rules. By modeling each class with a Gaussian distribution, posterior probabilities and decision boundaries could be calculated in a clear and interpretable way. The results also showed that adding a misclassification penalty can meaningfully change classification behavior by shifting the decision threshold. Finally, comparing features demonstrated that sepal length is a better choice than sepal width for separating Setosa and Versicolour, since it produces clearer boundaries and more confident decisions.